Chapter 3
Search Algorithms and Applications

Asst.Prof.Rapeeporn Chamchong, PhD

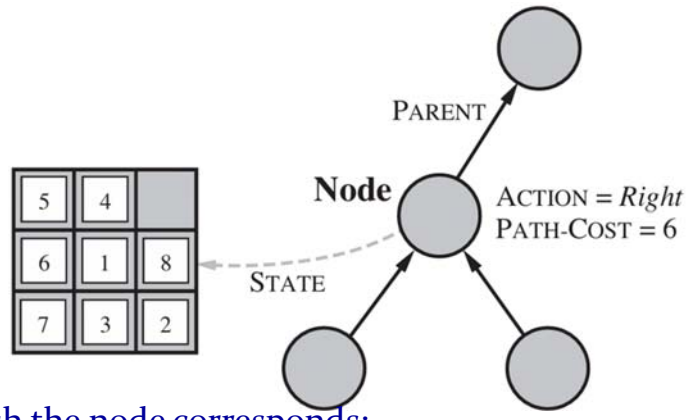
Searching Strategies

- Blind/uninformed
  - Breath-first Search (BFS)
  - Depth-first Search (DFS)
  - Depth-Limit Search (DLS)
  - Iterative Deepening Search (IDS)
- Informed/heuristic
  - Hill Climbing Search
  - Best First Search
  - Greedy Best First Search
  - A* Search

Search Algorithms

- All the problems that we have looked at can be converted to a form where we have to start from a start/initial state and search for a goal state by traveling through a search space.
- The search algorithms superimpose a search tree over the state space graph, forming various paths from the initial state, trying to find a path that reaches a goal state.
- Each node in the search tree corresponds to a state in the state space, and the edges in the search tree correspond to actions.
- The root of the tree corresponds to the initial state of the problem.

Search Data Structures



- A data structure is required to keep track of the search tree that is being constructed. For each node n of the tree that contains 4 components:
  - n.STATE: the state in the state space to which the node corresponds;
  - n.PARENT: the node in the search tree that generated this node;
  - n.ACTION: the action that was applied to the parent to generate the node;
  - n.PATH-COST: the cost, traditionally denoted by $g(n)$, of the path from the initial state to the node, as indicated by the parent pointers.
- Open list/ frontier (Use queue such as FIFO, LIFO/ stack, Priority Queue)
- Closed list/ reached/ visited

Uninformed Search

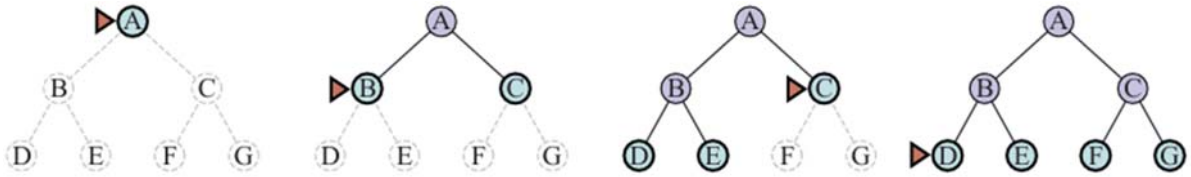
Breath-first Search (BFS)

Depth-first Search (DFS)

Depth-Limit Search (DLS)

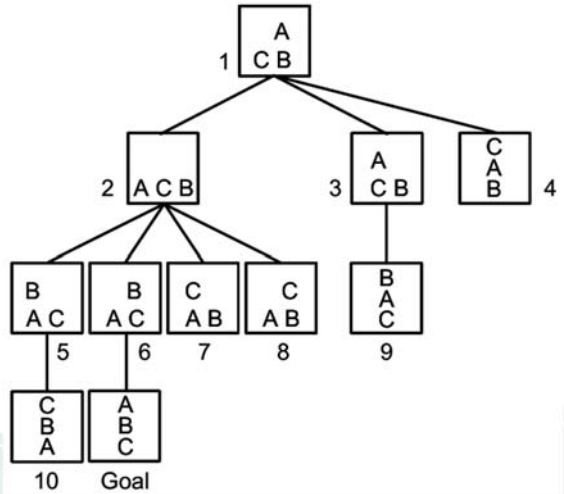
Iterative Deepening Search (IDS)

Breath First Search (BFS)



BFS on a simple binary tree. The node is expanded next by a marker.

BFS on a tree of block problem.



BFS Algorithm (Tree structure)

```
FUNCTION BFS_Search (Root_Node, Goal_State)
    CREATE Queue Frontier                                   // ใช้ Queue ทำงานแบบ FIFO
    CREATE Set Visited_Nodes
    IF Root_Node is NULL THEN                               // เริ่มต้นการค้นหา (Start Search)
        RETURN "Tree is Empty"
    END IF
    Frontier.Enqueue(Root_Node)                             // เพิ่มโหนดเริ่มต้นเข้าคิวและทำเครื่องหมายว่าถูกเยี่ยมชมแล้ว
    Visited_Nodes.Add(Root_Node)
    WHILE Frontier is NOT empty DO                          // วนลูปการค้นหา (Search Loop)
        Current_Node = Frontier.Dequeue()                   // ดึงโหนดที่อยู่หน้าสุดในคิวออกมา
        // ตรวจสอบเป้าหมาย (Goal Check)
        IF Current_Node equals Goal_State THEN
            RETURN "Goal Found at Current_Node"
        END IF
        FOR EACH Child_Node of Current_Node DO              // Expand Node สำรวจโหนดลูก (Neighbors/Children) ของ Current_Node
            IF Child_Node is NOT in Visited_Nodes THEN      // ถ้าโหนดลูกยังไม่เคยถูกเยี่ยมชม
                Visited_Nodes.Add(Child_Node)               // ทำเครื่องหมายว่าถูกเยี่ยมชมแล้ว
                Frontier.Enqueue(Child_Node)                // เพิ่มโหนดลูกเข้าคิวเพื่อรอการสำรวจในระดับถัดไป
            END IF
        END FOR
    END WHILE
    RETURN "Goal Not Found"                                 // ไม่พบเป้าหมาย (Failure)
END FUNCTION
```


BFS Algorithm (Graph structure)

```
FUNCTION BFS_Graph_Search (Graph, Start_Node, Goal_State)
    CREATE Queue Frontier
    CREATE Set Visited_Nodes                                // โครงสร้างข้อมูลแบบ FIFO, เพื่อติดตามโหนดที่เยี่ยมชมแล้ว
    IF Start_Node is NOT in Graph THEN                      //  เริ่มต้นการค้นหา (Start Search)
        RETURN "Start Node not found in Graph"
    END IF
    Frontier.Enqueue(Start_Node)                            // เพิ่มโหนดเริ่มต้นเข้าคิวและเช็คว่าเยี่ยมชมแล้ว
    Visited_Nodes.Add(Start_Node)
    WHILE Frontier is NOT empty DO                          // วนลูปการค้นหา (Search Loop)
        Current_Node = Q.Dequeue()                          // ดึงโหนดที่อยู่หน้าสุดในคิวออกมา
        IF Current_Node equals Goal_State THEN              // ตรวจสอบเป้าหมาย (Goal Check)
            RETURN "Goal Found at Current_Node"             // หากต้องการ Path, สามารถย้อนรอยจากโหนดนี้กลับไปที่ Start_Node ได้
        END IF
        FOR EACH Neighbor_Node in Get_Neighbors(Current_Node, Graph) DO  // Expand Node สำรวจเพื่อนบ้านทั้งหมดของ Current_Node
            IF Neighbor_Node is NOT in Visited_Nodes THEN   // ตรวจสอบว่าเพื่อนบ้านยังไม่เคยถูกเยี่ยมชม
                Visited_Nodes.Add(Neighbor_Node)            // ทำเครื่องหมายว่าถูกเยี่ยมชมแล้ว
                Frontier.Enqueue(Neighbor_Node)             // เพิ่มเพื่อนบ้านเข้าคิวเพื่อรอการสำรวจ
                // Set_Parent(Neighbor_Node, Current_Node)  // (Optional: เก็บข้อมูล Parent Pointer เพื่อใช้สร้างเส้นทางย้อนหลัง)
            END IF
        END FOR
    END WHILE
    RETURN "Goal Not Found"                                 // ไม่พบเป้าหมาย (Failure)
END FUNCTION
```

Comparison of Graph and Tree for BFS

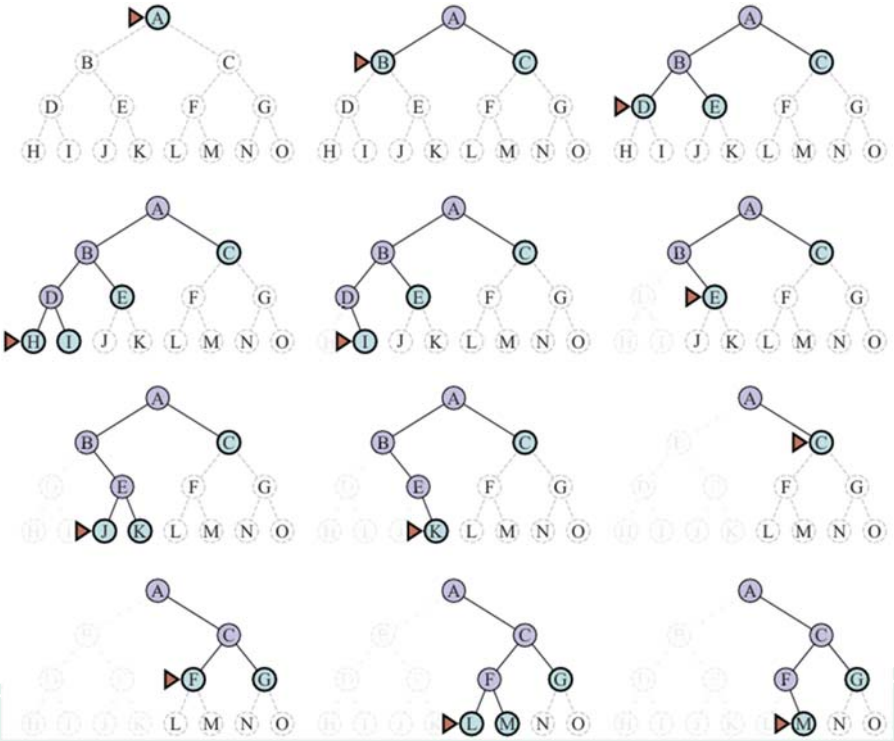
Attributes	General Graph BFS	Tree BFS
Visited_Noted	Required (Set) to prevent looping while traversing nodes, as graphs can have backtracking or overlapping paths.	Not Required because the tree structure does not process the same node repeatedly.
Neighbors	Children nodes are called neighbors or adjacent nodes. You must loop through the List/Set.	Only check child nodes at the same level.
Time Complexity	$O(V + E)$, $V = \text{Vertice}, E = \text{Edge}$	$O(N)$, $N = \text{Number of nodes}$

Importance of BFS in AI

- Completeness: BFS guarantees that it will always find the goal or answer if it exists in the state space.
- Optimality: Because BFS explores hierarchically, it guarantees that it will find the shortest path (with the fewest number of steps) to the goal. This is crucial for AI tasks such as pathfinding in unweighted graphs or solving state puzzles.

- BFS is therefore one of the most popular uninformed search techniques for solving basic AI tasks, such as:
  - Pathfinding: Finding the shortest path in a graph, such as a map or a robot's path.
  - 8-Puzzle: Finding the sequence of digits to reach the goal state.
  - Planning Problems: Finding the sequence of actions leading to a goal, such as a block world.

Depth-first Search (DFS)-Backtracking search

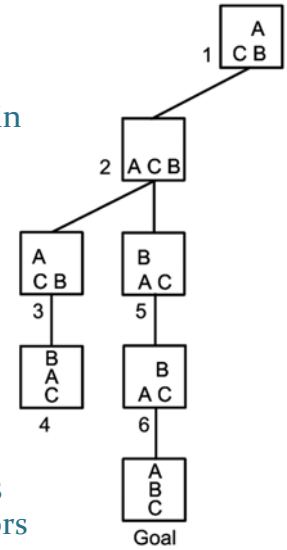


DFS on a tree.

The unexplored region is shown in light gray.

Explored nodes with no descendants in the frontier are removed from memory.

Nodes at depth 3 have no successors and *M* is the only goal node.



DFS Algorithm (Graph structure)

```
FUNCTION DFS_Search (Root_Node, Goal_State)
    CREATE Stack Frontier
    CREATE Set Visited_Nodes                    // DFS บนต้นไม้ โหนดจะไม่วนซ้ำ แต่ใช้กับ Graph เพื่อไม่ให้วนซ้ำ
    IF Root_Node is NULL THEN                   // เริ่มต้นการค้นหา (Start Search)
        RETURN "Tree is Empty"
    END IF
    Frontier.Push(Root_Node)                    // เพิ่มโหนดเริ่มต้นเข้าสแตก
    Visited_Nodes.Add(Root_Node)                // ทำเครื่องหมายว่าถูกเยี่ยมชมแล้ว
    WHILE Frontier is NOT empty DO              // วนลูปการค้นหา (Search Loop)
        Current_Node = Frontier.Pop()           // ดึงโหนดที่อยู่บนสุดในสแตกออกมา (LIFO)
        IF Current_Node equals Goal_State THEN  // ตรวจสอบเป้าหมาย (Goal Check)
            RETURN "Goal Found at Current_Node"
        END IF
        FOR EACH Child_Node in REVERSE_ORDER of Current_Node's Children DO    // Expand Node สำรวจโหนดลูก
            IF Child_Node is NOT in Visited_Nodes THEN  // ในโครงสร้างต้นไม้ โหนดลูกมักจะยังไม่ถูกเยี่ยมชมเสมอ
                Visited_Nodes.Add(Child_Node)   // ทำเครื่องหมายว่าถูกเยี่ยมชมแล้ว
                Frontier.Push(Child_Node)       // เพิ่มโหนดลูกเข้าสแตก (โหนดลูกนี้จะถูกสำรวจถัดไปทันที)
            END IF
        END FOR
    END WHILE
    RETURN "Goal Not Found“                     // ไม่พบเป้าหมาย (Failure)
END FUNCTION
```

DFS Algorithm (Graph structure)

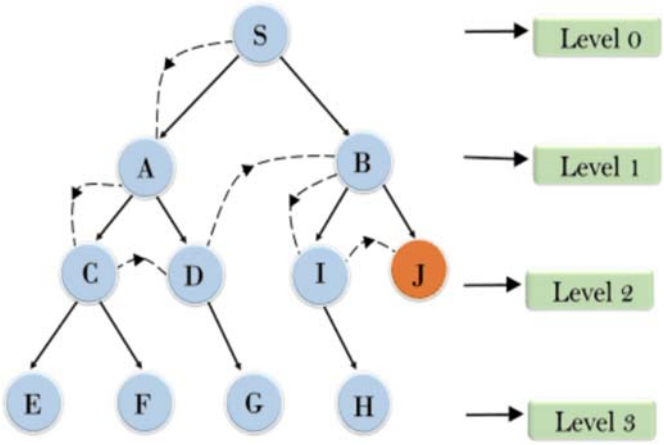
```
FUNCTION DFS_Graph_Search (Graph, Start_Node, Goal_State)
    CREATE Stack Frontier                                   // ใช้ Stack สำหรับการทำงานแบบ LIFO
    CREATE Set Visited_Nodes                                // เพื่อติดตามโหนดที่เยี่ยมชมแล้ว (สำคัญมากสำหรับ Graph)
    IF Start_Node is NOT in Graph THEN                      // เริ่มต้นการค้นหา (Start Search)
        RETURN "Start Node not found in Graph"
    END IF
    Frontier.Push(Start_Node)                               // เพิ่มโหนดเริ่มต้นเข้าสแตก
    Visited_Nodes.Add(Start_Node)                           // ทำเครื่องหมายว่าถูกเยี่ยมชมแล้ว
    WHILE Frontier is NOT empty DO                          // วนลูปการค้นหา (Search Loop)
        Current_Node = Frontier.Pop()                       // ดึงโหนดที่อยู่บนสุดในสแตกออกมา
        IF Current_Node equals Goal_State THEN              // ตรวจสอบเป้าหมาย (Goal Check)
            RETURN "Goal Found at Current_Node"
        END IF
        FOR EACH Neighbor_Node in REVERSE_ORDER of Get_Neighbors(Current_Node, Graph) DO  // Expand Node สำรวจเพื่อนบ้าน
            IF Neighbor_Node is NOT in Visited_Nodes THEN   // ตรวจสอบว่าเพื่อนบ้านยังไม่เคยถูกเยี่ยมชม
                Visited_Nodes.Add(Neighbor_Node)            // ทำเครื่องหมายว่าถูกเยี่ยมชมแล้ว
                Frontier.Push(Neighbor_Node)                // เพิ่มเพื่อนบ้านเข้าสแตก (โหนดนี้จะสำรวจทันทีตามความลึก)
            END IF
        END FOR
    END WHILE
    RETURN "Goal Not Found“ // ไม่พบเป้าหมาย (Failure)
END FUNCTION
```


DFS Properties in AI

1) Completeness: DFS on trees is incomplete if the tree has infinite depth, as the search will be stuck on infinite paths. However, it is complete if the search space has a finite depth.
2) Shortest Path Finding: DFS does not guarantee the shortest path, but always finds the first solution.
3) Applications: It is suitable for problems where we know the solution lies at a large depth, or where the search space has a high branching factor but limited depth, such as game tree generation or the exploration of very deep data structures.

Depth-Limit Search: DLS (การค้นหาแบบจำกัดความลึก)

- Solves DFS as an infinite depth path problem (l=∞).
- Set predefined depth limit.
- Nodes at the limit might not have successors.
- Optimal limit often unknown beforehand.
- Implement using a recursive algorithm.
- Efficiency & performance
  - Time complexity: $O(b^l)$
  - Space complexity: $O(b \times l), high$
  - Optimality: might not have successor
  - Completeness: Not complete if $l < d$, even $l > d$.
  - b =branch factor, l = level and d = depth



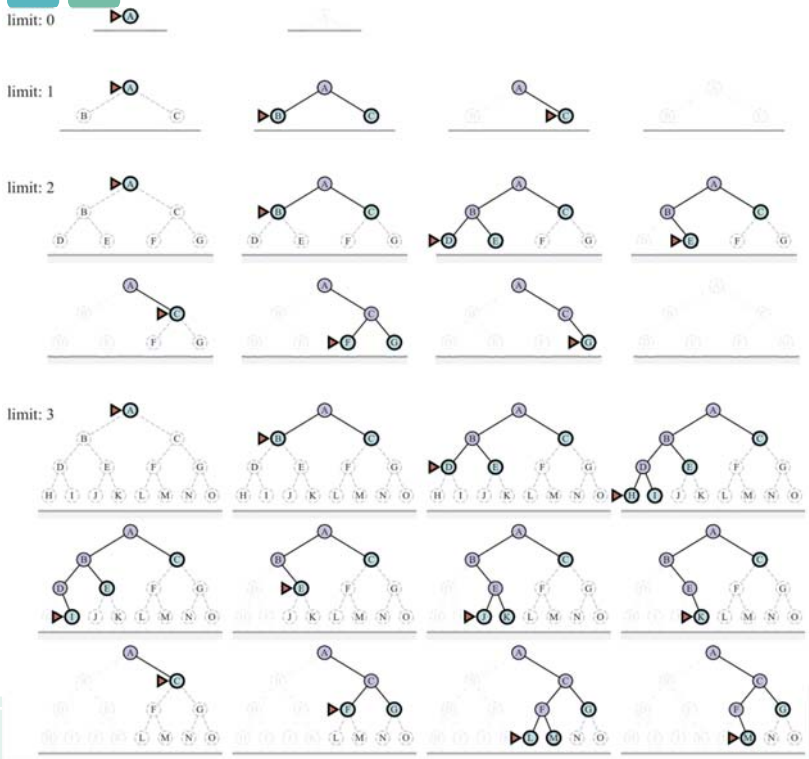
DLS Algorithm

```
FUNCTION DLS_Search (Graph, Start_Node, Goal_State, Limit)
    CREATE Set Visited_Node // ป้องกันการวนซ้ำ
    Result = Recursive_DLS(Graph, Start_Node, Goal_Node, Limit, 0, Visited)
    IF Result is NOT NULL THEN
        RETURN Result // พบเป้าหมาย
    ELSE IF Result equals "Cutoff" THEN
        // ถึงขีดจำกัดความลึก แต่ไม่พบเป้าหมาย
        RETURN "Failure: Cutoff occurred"
    ELSE
        RETURN "Goal Not Found" // สำรวจจนหมดและไม่พบเป้าหมาย
    END IF
END FUNCTION
```

```
FUNCTION Recursive_DLS (Graph, Current_Node, Goal_Node, Limit, Current_Depth, Visited)
  IF Current_Node equals Goal THEN          // 1. ตรวจสอบเป้าหมาย (Goal Check)
    RETURN Current_Node // พบเป้าหมายแล้ว
  END IF
  IF Current_Depth equals Limit THEN        // 2. ตรวจสอบขีดจำกัดความลึก (Depth Limit Check)
    RETURN "Cutoff"                          // ค้นหาถึงขีดจำกัดแล้ว ให้หยุดสำรวจเส้นทางนี้
    END IF
  SET Cutoff_Occurred = FALSE               // 3. เตรียมการสำรวจ
  Visited_Nodes.Add(Current_Node)           // ทำเครื่องหมายโหนดปัจจุบันว่าถูกเยี่ยมชมแล้ว
  FOR EACH Neighbor in Get_Neighbors(Current_Node, Graph) DO
    // 4. ขยายโหนดและเรียกตัวเองซ้ำ
    IF Neighbor is NOT in Visited_Nodes THEN
      // เรียกตัวเองซ้ำเพื่อลงลึกไปอีกระดับ
      Result = Recursive_DLS(Graph,Neighbor, Goal_Node, Limit, Current_Depth + 1, Visited_Nodes)
      IF Result equals "Cutoff" THEN        // ตรวจสอบผลลัพธ์
        SET Cutoff_Occurred = TRUE          // mark ว่าถึงขีดจำกัด
      ELSE IF Result is NOT NULL THEN
        RETURN Result                       // ส่งผลลัพธ์การพบเป้าหมายกลับขึ้นไป
      END IF
    END IF
  END FOR

  // 5. สรุปผลลัพธ์ของโหนดนี้
  IF Cutoff_Occurred is TRUE THEN
    RETURN "Cutoff"                         // ส่งสถานะ Cutoff ขึ้นไป
  ELSE
    RETURN NULL                             // ไม่พบเป้าหมายและไม่เกิด Cutoff
  END IF
END FUNCTION
```

Iterative Deepening Search: IDS (การค้นหาแบบการวนซ้ำความลึก)



- Solve the problem of DLS as a limited depth by applying DFS & BFS.
- Completeness: Guaranteed to find the goal.
- Optimality: Shortest path.
- Time complexity: $O(b^d)$
- Space complexity: $O(bd)$, high
- b = branch factor, d=depth

IDS Algorithm

```
FUNCTION IDS_Search (Graph, Start_Node, Goal_Node, Max_Depth)
// วนลูปเพิ่มขีดจำกัดความลึก (d) ตั้งแต่ 0 ขึ้นไป
  FOR depth FROM 0 TO Max_Depth DO
    // 1. การเตรียมการ
    // ต้องสร้าง Visited_Nods ใหม่ทุกครั้งที่ความลึกเปลี่ยน เพื่อให้โหนดที่
    // ถูก Cutoff ไปแล้วสามารถถูกสำรวจได้อีกครั้งในความลึกที่มากขึ้น
    CREATE Set Visited_Nodes
    // 2. เรียกใช้ DLS
    Result = Recursive_DLS(Graph, Start_Node, Goal_Node, depth, 0, Visited_Nodes)
    // 3. ตรวจสอบผลลัพธ์
    IF Result is NOT "Cutoff" AND Result is NOT NULL THEN
      // พบเป้าหมาย หรือการค้นหาล้มเหลวโดยสมบูรณ์ (Failure)
      RETURN Result
    END IF
  END FOR
  // หากสำรวจถึง Max_Depth แล้วยังไม่พบ
  RETURN "Goal Not Found"
END FUNCTION
```

```
FUNCTION Recursive_DLS (Graph, Current_Node, Goal_Node, Limit, Current_Depth, Visited)
  IF Current_Node equals Goal THEN          // 1. ตรวจสอบเป้าหมาย (Goal Check)
    RETURN Current_Node // พบเป้าหมายแล้ว
  END IF
  IF Current_Depth equals Limit THEN        // 2. ตรวจสอบขีดจำกัดความลึก (Depth Limit Check)
    RETURN "Cutoff"                          // ค้นหาถึงขีดจำกัดแล้ว ให้หยุดสำรวจเส้นทางนี้
    END IF
  SET Cutoff_Occurred = FALSE               // 3. เตรียมการสำรวจ
  Visited_Nodes.Add(Current_Node)           // ทำเครื่องหมายโหนดปัจจุบันว่าถูกเยี่ยมชมแล้ว
  FOR EACH Neighbor in Get_Neighbors(Current_Node, Graph) DO
    // 4. ขยายโหนดและเรียกตัวเองซ้ำ
    IF Neighbor is NOT in Visited_Nodes THEN
      // เรียกตัวเองซ้ำเพื่อลงลึกไปอีกระดับ
      Result = Recursive_DLS(Graph,Neighbor, Goal_Node, Limit, Current_Depth + 1, Visited_Nodes)
      IF Result equals "Cutoff" THEN        // ตรวจสอบผลลัพธ์
        SET Cutoff_Occurred = TRUE          // mark ว่าถึงขีดจำกัด
      ELSE IF Result is NOT NULL THEN
        RETURN Result                       // ส่งผลลัพธ์การพบเป้าหมายกลับขึ้นไป
      END IF
    END IF
  END FOR
  // 5. สรุปผลลัพธ์ของโหนดนี้
  IF Cutoff_Occurred is TRUE THEN
    RETURN "Cutoff"                         // ส่งสถานะ Cutoff ขึ้นไป
  ELSE
    RETURN NULL                             // ไม่พบเป้าหมายและไม่เกิด Cutoff
  END IF
  // (Optional: ลบ Current_Node ออกจาก Visited_Nodes
  Visited.Remove(Current_Node)
END FUNCTION
```

Efficiency Comparison of Uninformed Search

Algorithm	Completeness	Optimality	Time	Space
BFS	Complete if branching factor (b) has a limit.	The optimal solution is the same cost for all actions (i.e., cost = 1).	$O(b^d)$	$O(b^d)$
DFS	Incomplete (may be infinitely deep).	Not guaranteed.	$O(b^m)$	$O(bm)$
DLS	If limit ≥ goal depth	The optimal solution is not the best solution.	$O(b^\ell)$	$O(b\ell)$
IDS	Complete	If the costs for all actions are the same.	$O(b^d)$	$O(bd)$

Summary of Uninformed Search

- BFS: Best in terms of completeness and optimality, but memory-intensive.
- DFS: Most memory-efficient, but incomplete and not optimal.
- DLS: Controls DFS performance for improved performance using limits, but not optimal.
- IDS: Combines the advantages of BFS and DFS, with completeness and optimality, and low memory consumption. It is ideal for many cases of data-free searches.

Informed Search/ Heuristic Search

Heuristic Evaluation Function

Hill Climbing Search

Best-First Search

Greedy Best-First Search

A* Search

Informed/ Heuristic based Search

- So far , we have looked into procedures that search the search space in an uninformed manner, which is costly with respect to either time, space, or both.
- We now focus on a few techniques that search the solution space in an informed manner using something which is called a heuristic.
- Such techniques are called heuristic-based searches or informed searches.

Informed/ Heuristic based Search

- The basic idea of a heuristic search is that rather than trying all possible search paths, you try and *focus on paths that seem to be getting you closer to your goal state* using some kind of a "guide".

Recall the example of the mouse searching for cheese. The smell of cheese guides the mouse in the maze; in other words, the strength of the smell informs the mouse how far it is from the goal state. Here, and it is quite accurate.

Heuristic

- You generally can't be sure that you are really near to your goal state in a search space.
- However, heuristics may provide *a good guess to move* in search space.
- Heuristic is some sort of guess/hint that can be represented by a numeric value.
  - It may represent winning chances in a game
  - It may represent the cost of a route in a map etc.
- It must be noted that heuristics don't always give us the right guess, and hence the correct solutions.
  - In, consider that the maze has fences fixed along some of the paths through which the smell can pass. Our heuristic might guide us on a path that is blocked by a fence; hence, again, the heuristic is misguiding us.

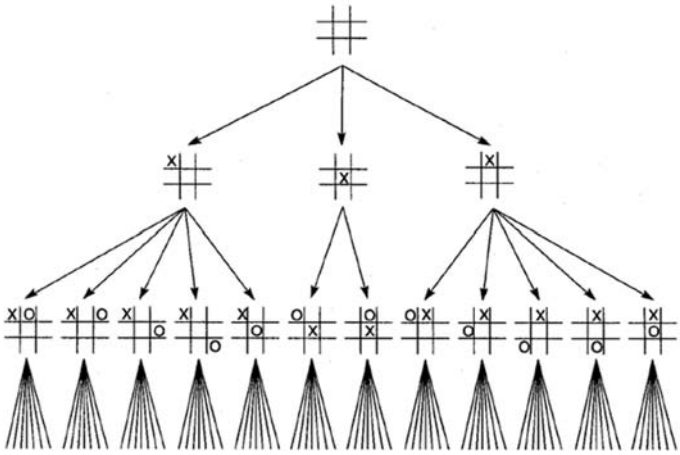
Heuristic

- Heuristic search is the search using knowledge to guide the path of the solution that
  - It is a technique to improve search efficiency, with a guide that may not find the correct answer even though the state space contains an answer.
  - It is useful to think of heuristic algorithms as consisting of two parts:
    - The heuristic function: To estimate the heuristic value. This value is calculated from the state that is close to the goal state.
    - An algorithm that uses it to search the state space.

Heuristic Evaluation Function

Heuristic Example-Tic-Tac-Toe

- In a complete search space there are 9! Possible state to search, through heuristics this search can be reduced. Symmetry reduction decreases the search space.



Heuristic Function

- Heuristic function: $h(n)$ is the cost path estimation from the current state.
- If the $h(n)$ is low, that means it is close to the goal state.
- If the $h(n)$ is high, that means it is far from the goal state.
- Heuristic function can be calculated from the Manhattan distance or the Euclidean distance.

Manhattan and Euclidean Distance

- Manhattan distance or city block distance
  - Calculate the distance in vertical and horizontal directions. (m is dimension)

$$D(p,q) = \sum_{i=1}^{m} |p_i - q_i|$$

  - Ex: Two points of two dimensions $A(2, 3)$, $B(5, 7)$

$$D(A,B) = |5-2|_x + |7-3|_y = 3 + 4 = 7$$

  - Suitable for grid or table.
- Euclidean distance
  - Calculate the distance by measuring between two points directly.

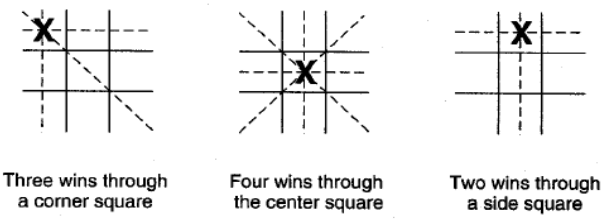
$$D(p,q) = \sqrt{\sum_{i=1}^{m} (p_i - q_i)^2}$$

  - Ex: Two points of two dimensions $A(2, 3)$, $B(5, 7)$

$$D(A,B) = \sqrt{(5-2)^2 + (7-3)^2} = \sqrt{9+16} = \sqrt{25} = 5$$

  - Suitable for continuous space (e.g. Similarity, Clustering)

Heuristic Example-Tic-Tac-Toe



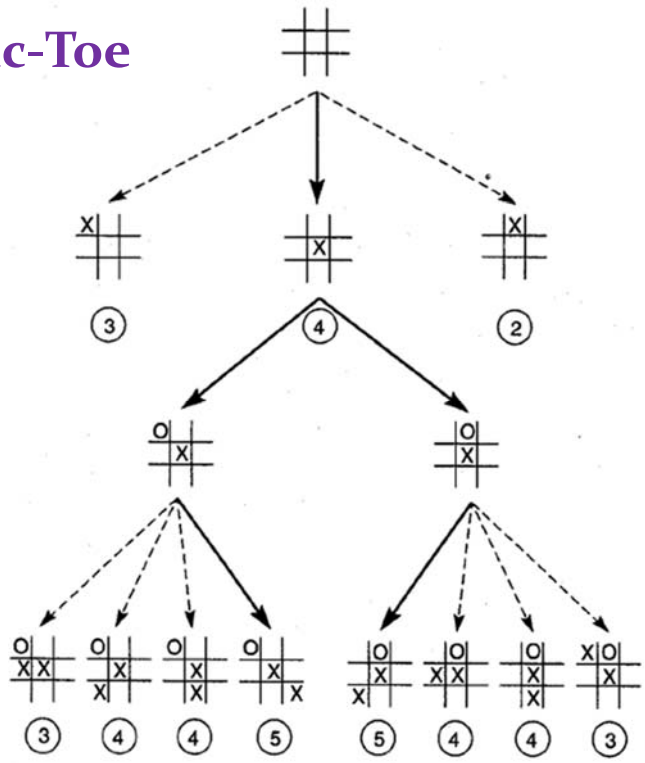
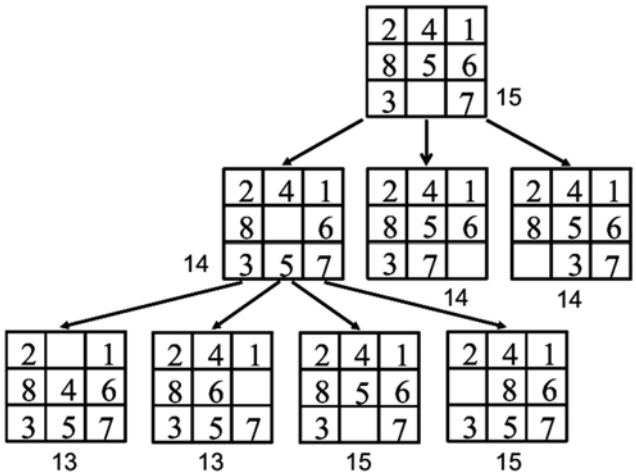


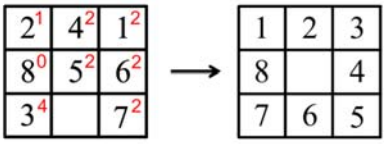
Figure Heuristically reduced state space for tic-tac-toe.

Heuristic Example-8 Puzzle

- Manhattan function (Manhattan distance)
  - $h(n) = \sum_{i=1}^{m} \sum_{j=1}^{n} |p_j - q_j|_i$
  - For 2D, $h(n) = \sum_{j=1}^{n} \left(|p_j - q_j|_x + |p_j - q_j|_y \right)$
  - Minimum of $h(n)$
- The first state: $n = 8$



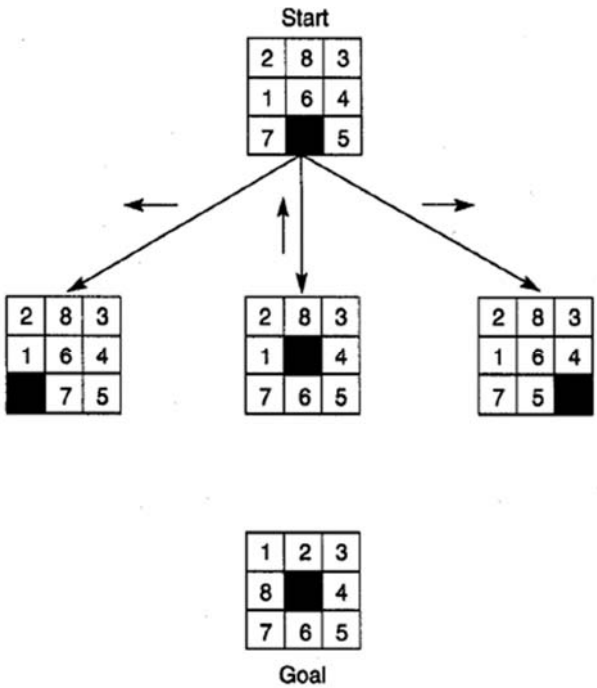
$h(n) = (1+0)+(1+1)+(2+0)+0+(1+1)+(1+1)+(2+2)+(2+0)=15$

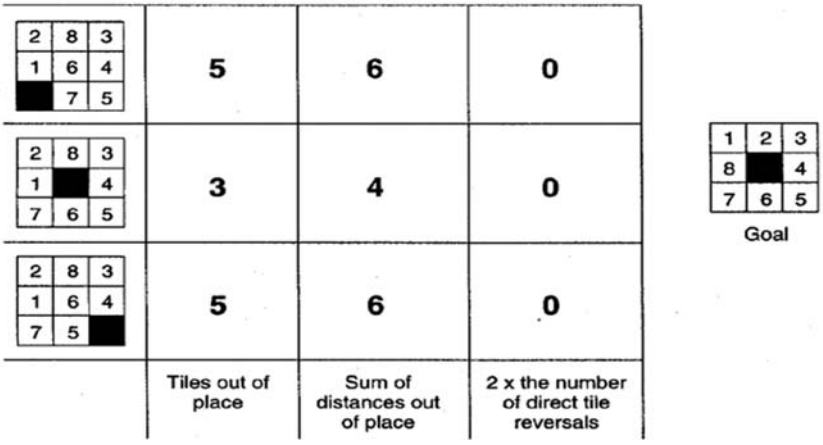


Start Goal

Source: บุญเสริม ถาวรกุล

Heuristic Example-8 Puzzle



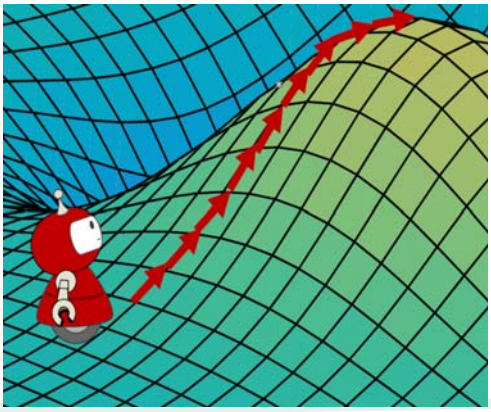


Informed Searches

- Searches that take place on the basis of some hints/ heuristics.
- It is also known as Heuristic Based Search
- In order to search on the basis of some heuristic, we must have estimated heuristic values to a certain desired level (not necessarily from the root node and similarly not till the leaf nodes)
- These methods may have access to a heuristic function $h(n)$ that estimates the cost of a solution from n
- Algorithm informed searches:
  - Hill Climbing
  - Best First Search
    - Greedy Best First Search
    - A* Search

Hill Climbing

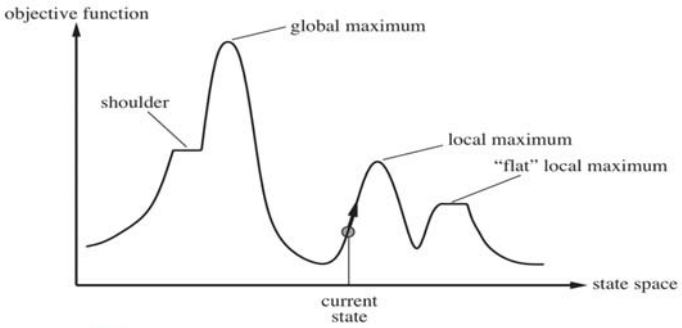
- Hill Climbing is basically a depth-first search with a measure of quality (cost/heuristic) that is assigned to each node in the tree.
- The basic idea is to traverse in DFS manner except that you order your choices according to some heuristic measurement of the remaining distance to the goal.

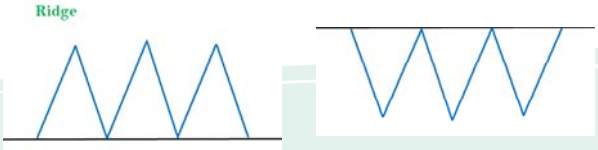


- Lets discuss an analogy for which the name Hill Climbing has been given to this procedure.
- Consider a blind person climbing a hill. He can not see the peak of the hill. The best way is to take steps from a given point; and steps in all possible directions. He finds that a step takes him higher and reaches a new, higher point. He continues this process until no possible steps will lead him higher, marking it as the peak; hence, the name hill climbing. Notice that each step that we take gets us closer to our goal, which, in this example, is the peak of a hill.
- Such a procedure might as well have some problems.

Problems of Hill Climbing

- **Foothill Problem**: Before reaching the global maximum, the blind man may find local maxima, which are intermediate peaks.
  - At these points, he might mistakenly believe he has reached the global maximum, as no steps take him higher.
  - Consequently, he may get stuck, thinking he has found the solution.
- **Plateau Problem**: In mountainous regions, flat areas called plateaus can cause a blind person to become stuck. On these flat sections, every step fails to change elevation, leaving them unable to progress.
- **Ridges**: A ridge is a specific type of local maximum that is higher than its surrounding regions, but it has a slope and cannot be reached in a single move.



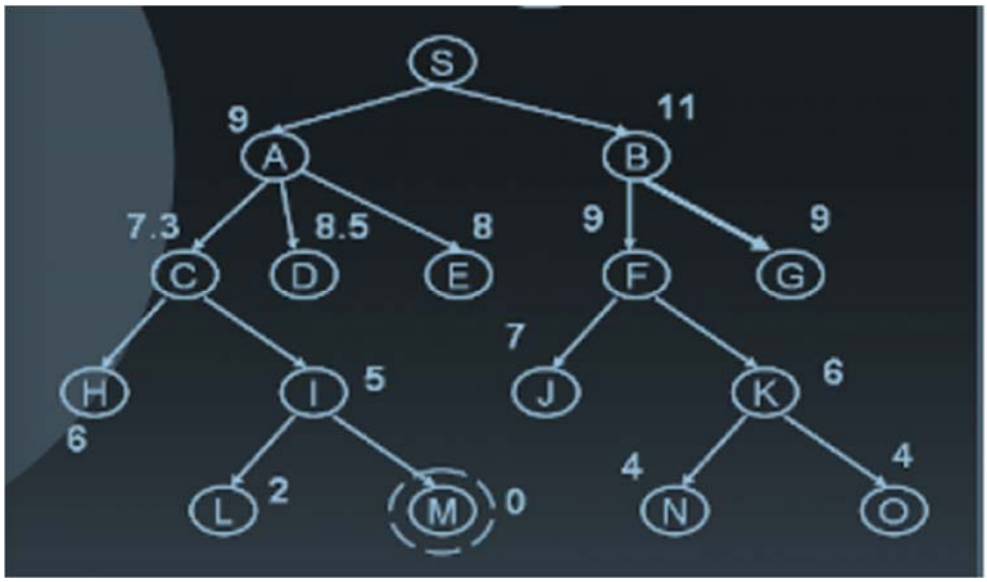


Hill Climbing Algorithm

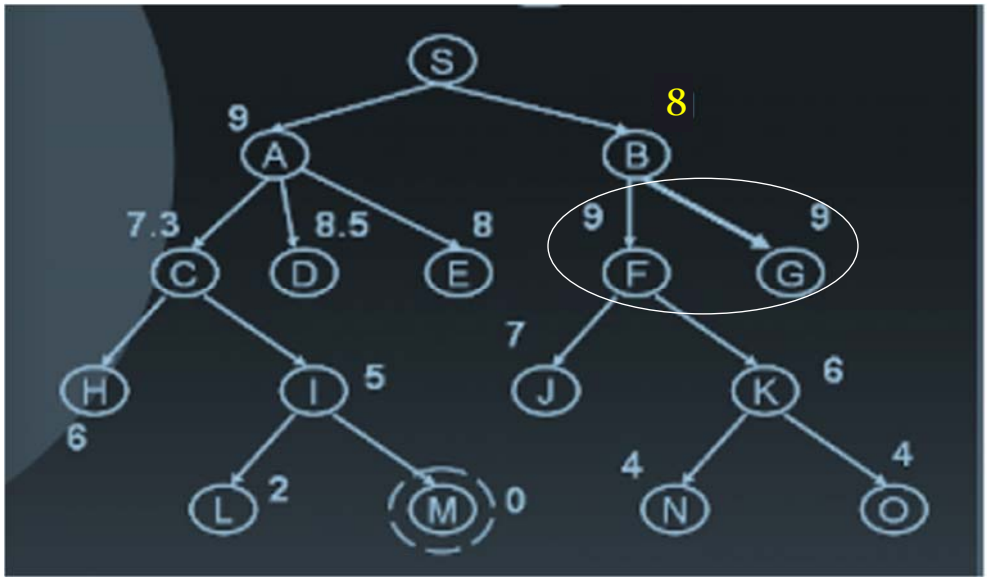
function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

```
current ← MAKE-NODE(problem.INITIAL-STATE)
loop do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE ≤ current.VALUE then return current.STATE
    current ← neighbor
```

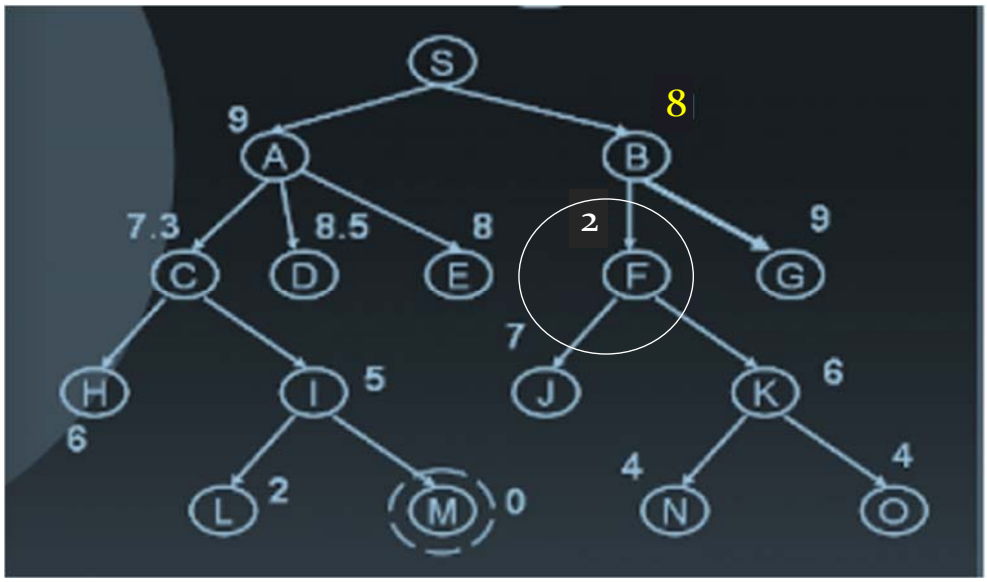
Hill Climbing Algorithm Example



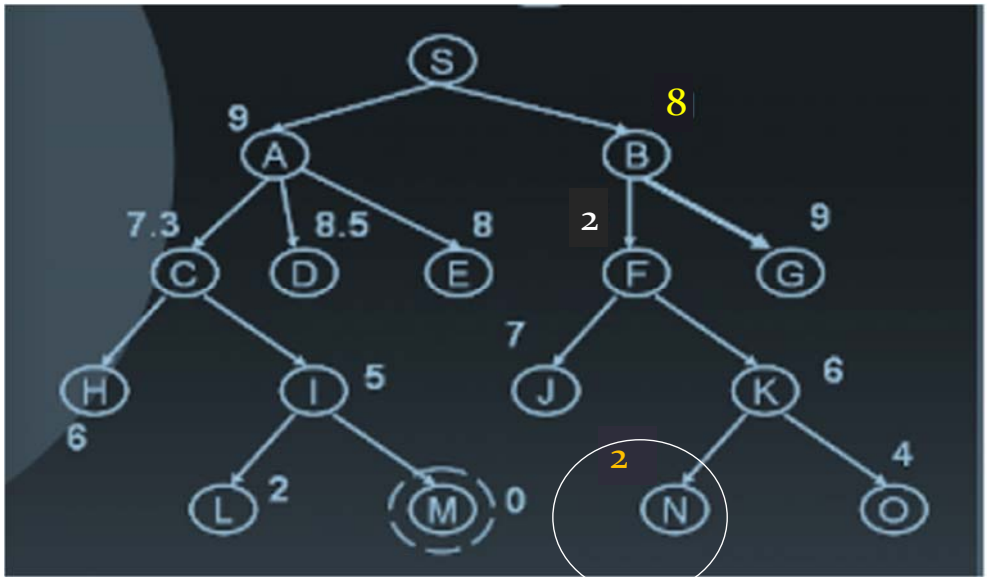
Hill Climbing Algorithm Example- Plateau



Hill Climbing Algorithm Example- Local Minima



Hill Climbing Algorithm Example- Ridges



Best-First Search (Greedy approach)

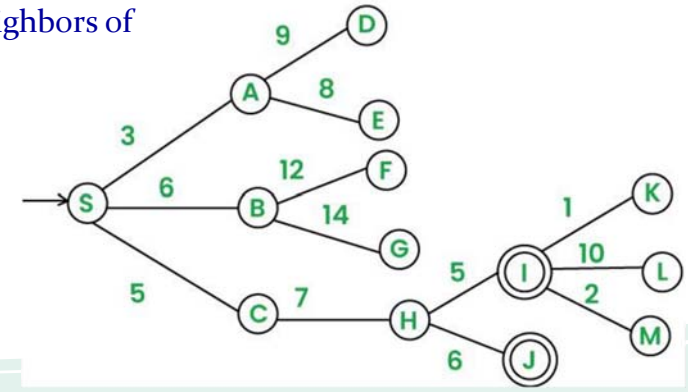
- Hence best first search is a greedy approach will looks for the best amongst the available options and hence can sometimes reduce the searching time.
- All these heuristically informed procedures are considered better but they do not guarantee the optimal solution, as they are dependent on the quality of heuristic being used.
- $f(n) = h(n)$

Greedy Best-First-Search Algorithm

```
FUNCTION GBFS_Search (Graph, Start_Node, Goal_State)
    CREATE Priority_Queue Frontier                                  // Priority Queue: เก็บโหนดที่สำรวจเรียงตามค่า h(n) รูปแบบข้อมูล: (h_value, Node)
    CREATE Set Visited_Nodes                                        // Set: เพื่อติดตามโหนดที่ถูกเยี่ยมชมแล้ว (ป้องกันการวนซ้ำ)
    H_Start = Heuristic(Start_Node, Goal_State)                     // คำนวณค่า h(n) สำหรับโหนดเริ่มต้น
    Frontier.Insert(H_Start, Start_Node)                            // เพิ่มโหนดเริ่มต้นเข้า Queue
    Visited_Nodes.Add(Start_Node)
    WHILE Frontier is NOT empty DO                                  // Search loop
        (Current_H, Current_Node) = Frontier.Extract_Min()          // ดึงโหนดที่มีค่า Heuristic น้อยที่สุดออกมา
        IF Current_Node equals Goal_State THEN                      // ตรวจสอบเป้าหมาย (Goal Check)
            RETURN "Goal Found at Current_Node"
        END IF
        FOR EACH Neighbor_Node in Get_Neighbors(Current_Node, Graph) DO   // Expand Node สำรวจเพื่อนบ้าน (Neighbors) ทั้งหมดของ Current_Node
            IF Neighbor_Node is NOT in Visited_Nodes THEN           // ตรวจสอบว่าเพื่อนบ้านยังไม่เคยถูกเยี่ยมชม
                Visited_Nodes.Add(Neighbor_Node)                    // ทำเครื่องหมายว่าถูกเยี่ยมชมแล้ว
                H_Neighbor = Heuristic(Neighbor_Node, Goal_State)   // คำนวณค่า h(n) สำหรับเพื่อนบ้าน
                Frontier.Insert(H_Neighbor, Neighbor_Node)          // เพิ่มเพื่อนบ้านเข้า Frontier โดยเรียงตาม H
                END IF
        END FOR
    END WHILE
    RETURN "Goal Not Found“                                         // ไม่พบเป้าหมาย (Failure)
END FUNCTION
```

Example Greedy Best-First-Search

- Start from source "S" and search for goal "I" using given costs and Best First search.
- Frontier initially contains S
  - Remove S from Frontier and process unvisited neighbors of S to Frontier.
  - Frontier now contains {A, C, B} (C is put before B because C has the lowest cost)
- Remove A from Frontier and process unvisited neighbors of A to Frontier
  - Frontier now contains {C, B, E, D}
- Remove C from Frontier and process unvisited neighbors of C to Frontier.
  - Frontier now contains {B, H, E, D}
- Remove B from Frontier and process unvisited neighbors of B to Frontier.
  - Frontier now contains {H, E, D, F, G}
- Remove H from Frontier.
- Since our goal "I" is a neighbor of H, then return.



Source: Geeksforgeeks

A* Search

- Heuristics are fallible; they can mislead.
- If two states have the same heuristics value at some middle phase, it does not mean that both will provide you the shortest path.
- Whenever such situations occur it is better to examine the state that is nearest to the root state of the graph.
- Among the two states having the same heuristic, the state whose distance is shorter from the start state will have a greater probability of being on the shortest path to the goal.

Heuristic Evaluation for A*

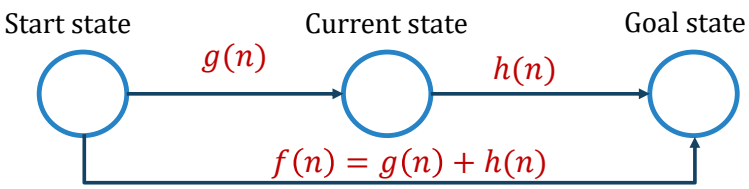
- The distance from the root can be measured by maintaining a depth count for each state. By accommodating this information our heuristic function will become as

$$f(n) = g(n) + h(n)$$

The $g(n)$ measures the distance from any state 'n' to the start state.

The $h(n)$ is a heuristic estimate of the distance from state n to a goal.

- After assigning heuristics value we can apply best-first algorithm to search for a goal.



A* Algorithm

```
FUNCTION A_STAR_Search (Graph, Start_Node, Goal_State)
    CREATE Priority_Queue Frontier                                  // Initialization รูปแบบข้อมูล: (f_value, g_value, Node, Parent_Node)
    CREATE Set Visited_Nodes                                        // Priority Queue เก็บโหนดที่กำลังจะสำรวจ เรียงตาม f(n) น้อยที่สุด
    CREATE Map G_Scores, Map Parents                                // Set/Dictionary: เก็บโหนดที่สำรวจเสร็จสิ้นแล้ว
    G_Scores[Start_Node] = 0                                        // Dictionary: เก็บค่า g(n) สำหรับโหนดที่พบแล้ว, และ: เก็บ Parent เพื่อใช้สร้างเส้นทางย้อนหลัง
    H_Start = Heuristic(Start_Node, Goal_State)                     // เริ่มต้นการค้นหา (Start Search)
    F_Start = 0 + H_Start      // คำนวณค่าเริ่มต้น f(Start) = g(Start) + h(Start)
    Frontier.Insert(F_Start, G_Scores[Start_Node], Start_Node, NULL)   // เพิ่มโหนดเริ่มต้นเข้า Frontier
    WHILE Frontier is NOT empty DO                                  // วนลูปการค้นหา (Search Loop)
        (Current_F, Current_G, Current_Node, Current_Parent) = Frontier.Extract_Min()   // ดึงโหนดที่มีค่า F น้อยที่สุดออกมา
        IF Current_Node equals Goal_State THEN                      // ตรวจสอบเป้าหมาย (Goal Check)
            RETURN "Goal Found", Reconstruction_Path(Parents, Current_Node)   // Reconstruction_Path(Parents, Current_Node) เป็นฟังก์ชันแยก
        END IF
        Visited_Nodes.Add(Current_Node)                             // เพิ่มโหนดปัจจุบันเข้า Closed List
        FOR EACH Neighbor_Node in Get_Neighbors(Current_Node, Graph) DO   // การขยายโหนด (Expand Node)
            IF Neighbor_Node is NOT in Frontier THEN                // ตรวจสอบว่าเพื่อนบ้านยังไม่ได้ถูกประมวลผลเสร็จสิ้นแล้ว
                Tentative_G_Score = Current_G + Cost(Current_Node, Neighbor_Node)   // คำนวณค่า g ใหม่
                IF Tentative_G_Score < G_Scores.Get(Neighbor_Node, INFINITY) THEN   // ตรวจสอบว่าเส้นทางใหม่นี้ดีกว่าเส้นทางที่เคยพบหรือไม่
                    Parents[Neighbor_Node] = Current_Node           // บันทึกเส้นทางที่ดีกว่า
                    G_Scores[Neighbor_Node] = Tentative_G_Score     // อัปเดตต้นทุน
                    H_Neighbor = Heuristic(Neighbor_Node, Goal_State)   // คำนวณค่า f ใหม่
                    F_Neighbor = Tentative_G_Score + H_Neighbor
                    IF Neighbor_Node is NOT in Frontier THEN        // ถ้า Neighbor ยังไม่ถูกเพิ่มใน Frontier (หรือถูกเพิ่มแล้วแต่กำลังจะอัปเดต)
                        Frontier.Insert(F_Neighbor, Tentative_G_Score, Neighbor_Node, Current_Node)
                    ELSE                                            // ถ้าอยู่ใน Frontier แล้ว ให้อัปเดตค่า F และ G
                        Frontier.Update(Neighbor_Node, F_Neighbor, Tentative_G_Score)
                    END IF
                END IF
            END IF
        END FOR
    END WHILE
    RETURN "Goal Not Found"
END FUNCTION
```

A* Example

- Frontier:[a4];
  Visited=[ ]
- Frontier:[c4, b6, d6];
  Visited=[a4]
- Frontier:[e5, f5, g6, b6, d6];
  Visited=[a4, c4]
- Frontier:[f5, h6, g6, b6, d6, i7];
  Visited=[a4, c4, e5]
- Frontier:[j5, h6, g6, b6, d6, k7, i7];
  Visited=[a4, c4, e5, f5]
- Frontier:[l5, h6, g6, b6, d6, k7, i7];
  Visited=[a4, c4, e5, f5, j5]
- Frontier:[m5, h6, g6, b6, d6, k7, i7];
  Visited=[a4, c4, e5, f5, j5, l5]
- Success, m= goal!

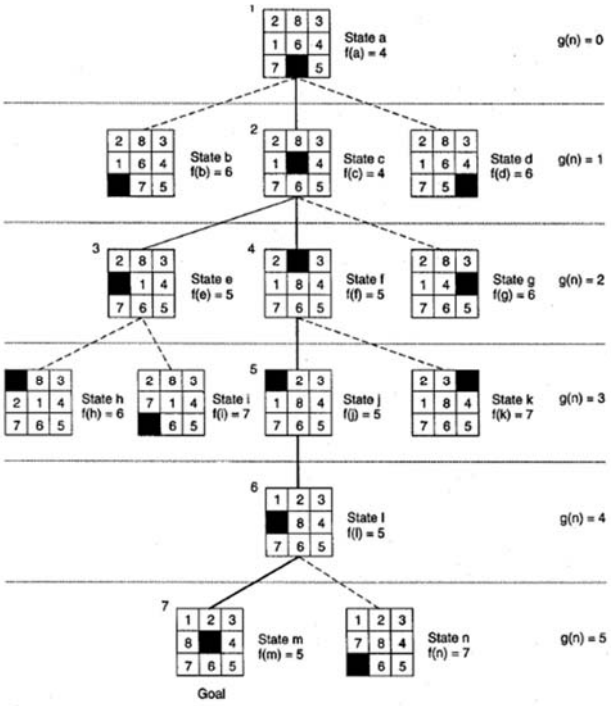
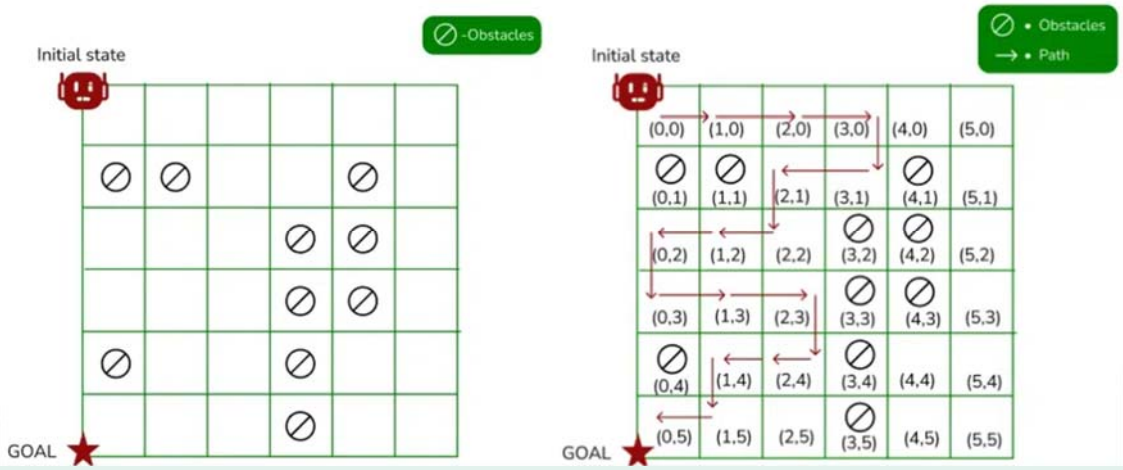


Figure State space generated in heuristic search of the 8-puzzle graph.

Assignment (Lab3)

- Solve the pathfinding problem of robotics using .
  - States: size=(6x6)
  - Initial state: (0,0)
  - Goal state: (0,5)
  - Obstacles: (0,1),(1,1),(3,2),(3,3),(3,4),(3,5), (0,4),(4,1),(4,2),(4,3)



Travelling Salesman Problem (TSP)

- **Objective**: Find the shortest possible route that allows a salesman to visit each city exactly once and return to the starting city.
- **Applications**: Logistics, routing, circuit design, and optimization problems.
- **Input**: A graph where nodes represent cities and edges represent the distance between them.

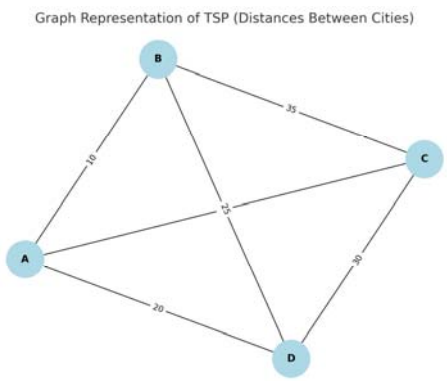
Greedy Approach to TSP

- The **greedy algorithm** for TSP builds a solution step-by-step by selecting the best immediate (local) choice at each step, with the hope that this leads to a global optimum.
- **Steps**:
  - **Start** at any city (typically the first city in the input).
  - From the current city, **select the nearest unvisited city** as the next destination.
  - Repeat this until all cities are visited.
  - Return to the starting city to complete the tour.

Pros & Cons of the Greedy Approach

- **Pros:**
  - **Simplicity**: Easy to understand and implement.
  - **Speed**: Works efficiently for smaller instances of the problem.
- **Cons:**
  - Not guaranteed to find the optimal solution: It often results in suboptimal solutions because it focuses on local optima.
  - For TSP, this approach does not consider the overall structure of the tour and may miss shorter paths.

Example

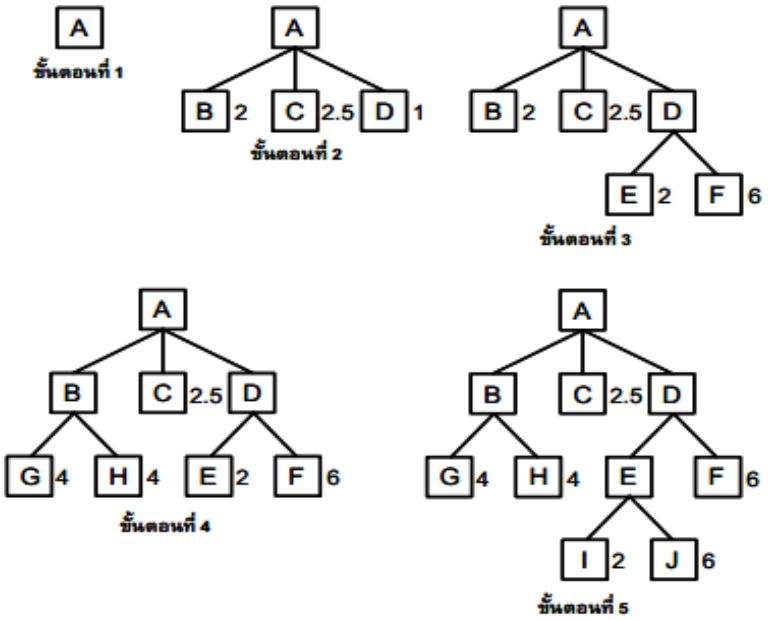


- For cities A, B, C, and D with the following distances:
  - Start at A.
  - Nearest unvisited city from A is B (distance = 10).
  - From B, go to D (distance = 25).
  - From D, go to C (distance = 30).
  - Return to A (distance = 15).

From/To	A	B	C	D
A	-	10	15	20
B	10	-	35	25
C	15	35	-	30
D	20	25	30	-

- **Path**: A → B → D → C → A
- **Total Distance**: 10 + 25 + 30 + 15 = 80.

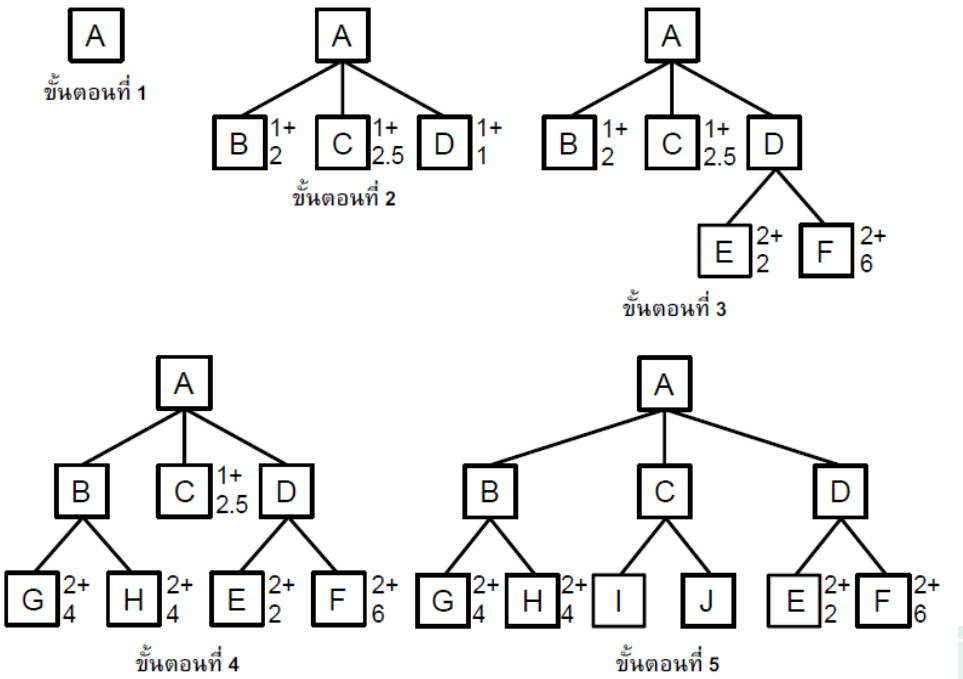
Greedy Best-First-Search



Assignment

Create the list of each state as the last example.

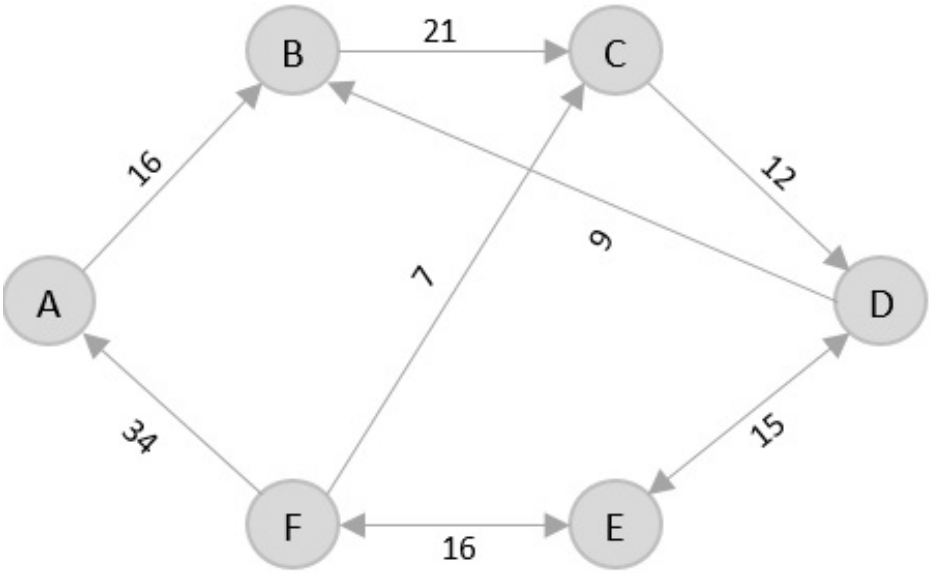
A*



Assignment 3

Create the list of each state as the last example.

Travel Salesman Problem



Assignment

Find the shortest path using greedy best-first search.

Comparison BFS & DFS

BFS	DFS
Use Queue data structure	Use Stack data structure
Build the tree level by level.	Build the tree sub-tree by sub-tree.
Work on the concept of FIFO	Work on the concept of LIFO
Suitable for searching vertices closer to the given source.	Suitable when there are solutions away from the source.
Requires a lot of memory	Uses less memory than BFS
Will not get stuck in a very deep path without finding the answer.	Get stuck in a very deep path without finding the answer.
If the answer is at n+1, all other states at level 1 to n will be spread.	If the answer is at n+1, all other states at level 1 to n won't be spread.
If the answer is found, it guarantees that it is the shortest path.	Even if an answer is found, it doesn't ensure that it is the shortest path.

Comparison Hill Climbing and Greedy

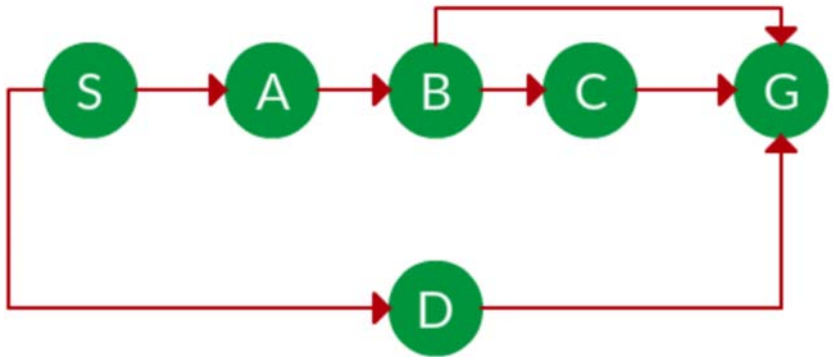
Greedy Best First Search	Hill Climbing
Choose the path with the lowest heuristic cost	Discover the highest point in the search space, even if it isn't the global maximum
Estimate the cost of getting to the target node using heuristics.	Evaluate nearby solutions using heuristics.
It doesn't have to keep track of prior nodes.	Only keeps track of the most recent and effective solutions
Not guaranteed to find a solution.	Not always possible to locate the global maximum.
Suitable heuristic, it is possible to locate a solution quickly in a wide search space.	Effective in locating a local maximum, but it can become trapped in a local optimum
Use BFS	Use DFS
Does not require backtracking.	It can backtrack steps if a better answer cannot be found.
Use in situations involving pathfinding and graph traversal.	Use in scheduling and logistics optimization problems.

Comparison Greedy and A*

Greedy Best First Search	A*
Cost function f(n)=h(n)	Cost function f(n)=h(n)+g(n)
Search algorithm does not involve past knowledge.	Search algorithm involves past knowledge.
It is not a complete search.	It is not a complete search.
It is not optimal as the path found may not be optimal.	It is optimal as the path found is always optimal.
The complexity of time is O(bm), and space can be polynomial. where b is the branching and m is the maximum depth of the search tree	Complexity of time is $O(b^m)$ and space is also $O(b^m)$. where b is the branching and m is the maximum depth of the search tree
Require less memory.	Require more memory.
It keeps all the fringe or border nodes in the memory while searching.	It keeps all the nodes in the memory while searching.

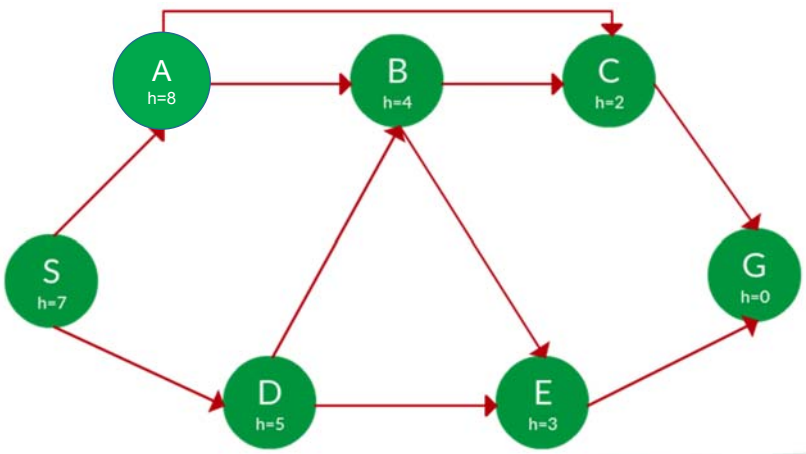
Problems

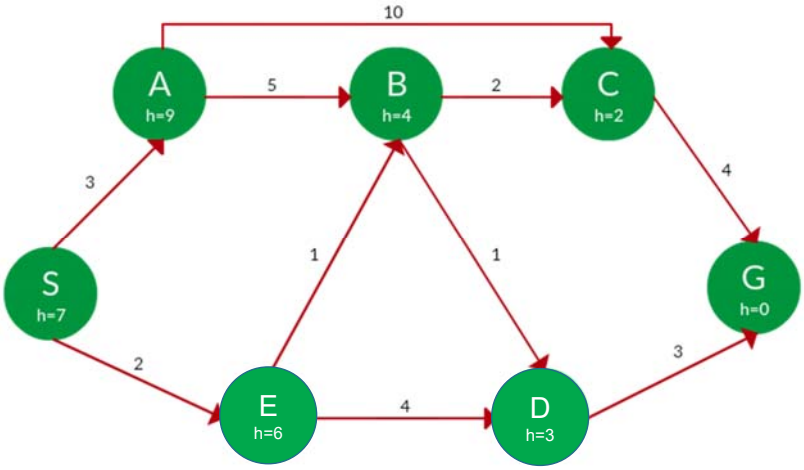
- Which solution would BFS and DFS find to move from node S to node G if run on the graph below?



Greedy Best First Search

- Find the path from S to G using greedy search. The heuristic values h of each node below the name of the node.



A* Search

- Find the path to reach from S to G using A* search.